

DSA V2 — Product-Based / USA Interview Master Plan

JavaScript-first • Beginner fundamentals → patterns → 100 core problems → advanced interview mastery

Target: Product-based engineering interviews, including USA-style technical interviews. This is an interview-preparation roadmap, not a promise of any company's exact interview questions.

One-thing-at-a-time rule: Finish the current stage and pass its mastery gate before moving on. Do not rush because a calendar says it is time.

1. The Complete Progression

- Stage 0 — JavaScript for DSA
- Stage 1 — Beginner programming + logic-building interview questions
- Stage 2 — Time & Space Complexity
- Stage 3 — DSA fundamentals and pattern recognition
- Stage 4 — 100 Core Product-Based Interview Problems
- Stage 5 — Pattern mastery: Easy → Medium → Hard → Blind
- Stage 6 — Advanced DSA + Dynamic Programming + Graphs
- Stage 7 — Striver A2Z completeness audit
- Stage 8 — NeetCode interview reinforcement
- Stage 9 — Codeforces contests and unseen problems
- Stage 10 — Company-specific sets + mock interviews

2. Where To Learn

Resource	Role
MDN Web Docs	JavaScript language/reference. Use when JS syntax or built-in behavior is unclear.
CodeWithHarry	Primary beginner-friendly JavaScript and initial DSA teaching.
Abdul Bari	Deep algorithm intuition, correctness and complexity. Use selectively for difficult concepts.
LeetCode	Primary interview problem platform.
NeetCode	Pattern-based interview reinforcement after core fundamentals are established.
Striver A2Z	Comprehensive DSA checklist and later gap audit.
Codeforces	Unseen-problem solving, speed and contest practice after fundamentals.

3. Stage 0 — JavaScript DSA Foundation

- Variables, primitive/reference types, operators, conditionals and loops
- Functions, parameters, return values, scope and closures at a practical level
- Arrays: indexing, traversal, mutation, copying, slice/splice, push/pop, shift/unshift
- Strings and common string operations
- Objects, Map and Set; when each is appropriate
- Destructuring, spread/rest and useful modern JS syntax
- Sorting with custom comparators
- Stack/queue implementation using arrays
- Basic recursion and call-stack understanding
- Input/output conventions on coding platforms
- Writing clean functions and test cases

Gate: Implement small programs from a blank JS file without following a video line-by-line.

4. Stage 1 — Beginner Interview Questions

- 1. Reverse a string
- 2. Check whether a string is a palindrome
- 3. Count vowels and consonants
- 4. Find the largest of three numbers
- 5. Find maximum and minimum in an array
- 6. Calculate sum and average of an array
- 7. Count frequency of elements
- 8. Remove duplicates from an array
- 9. Find the second-largest element
- 10. Check whether an array is sorted
- 11. Reverse an array in-place
- 12. Rotate an array
- 13. Find a missing number from 1..n
- 14. Find duplicate values
- 15. Count occurrences of a target
- 16. Linear search
- 17. Binary search
- 18. Find first/last occurrence
- 19. Find factorial
- 20. Generate Fibonacci numbers
- 21. Check prime
- 22. List primes up to n
- 23. Find GCD
- 24. Find LCM
- 25. Reverse an integer
- 26. Check numeric palindrome
- 27. Sum digits

- 28. Count digits
- 29. Armstrong number
- 30. Power/exponentiation
- 31. FizzBuzz
- 32. Swap two values
- 33. Print common loop patterns
- 34. Print a matrix
- 35. Transpose a matrix
- 36. Find diagonal sums
- 37. Search a matrix
- 38. Merge two sorted arrays
- 39. Intersection of two arrays
- 40. Union of two arrays
- 41. Find majority element
- 42. Move zeroes
- 43. Basic frequency-map problem
- 44. Implement a stack
- 45. Implement a queue
- 46. Balanced parentheses
- 47. Recursive factorial
- 48. Recursive Fibonacci
- 49. Recursive binary search
- 50. Basic linked-list traversal
- 51. Insert/delete linked-list node
- 52. Reverse a linked list
- 53. Tree preorder/inorder/postorder traversal
- 54. Tree height
- 55. BFS level traversal

Gate: Solve most basic questions independently and explain your approach before coding.

5. Stage 2 — Time & Space Complexity

- $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$, $O(n!)$
- Sequential vs nested loops
- Sorting and searching complexity
- Array, object, Map and Set operation complexity at a practical interview level
- Recursion and call-stack space
- Auxiliary space vs input space
- Best, average and worst-case reasoning where relevant
- Trade-offs: time vs memory

Gate: Analyze 20 unseen snippets and defend your time and auxiliary-space complexity.

6. Stage 3 — Pattern Recognition Map

Pattern	Recognize when...
Arrays	The data is a sequence and the main work is traversal, rearrangement or maintaining array state.
Hashing	You need fast membership, frequency, counting, grouping or complement lookup.
Two Pointers	Two positions can move through ordered data to eliminate unnecessary work.
Sliding Window	The problem concerns a contiguous range and a maintained window can replace repeated calculation.
Prefix Sum	Cumulative information can answer repeated range/subarray questions.
Binary Search	The search space is sorted or monotonic and half can be eliminated.
Linked List	The structure is pointer-based and node connections must be manipulated.
Stack	LIFO, nested structure, previous/next relationships or monotonic candidates matter.
Queue/Deque	Processing order or a moving maximum/minimum needs FIFO/dequeue behavior.
Recursion	A problem naturally decomposes into smaller instances of itself.
Backtracking	You must explore choices, undo choices and enumerate valid configurations.
Tree DFS/BFS	Hierarchical structure requires recursive state or level-by-level traversal.
Heap	You repeatedly need the smallest/largest/best K candidate.
Intervals	Ranges overlap, merge, schedule or compete for resources.
Greedy	A locally optimal choice can be proven safe for the global objective.
Graph BFS/DFS	Entities form nodes/edges and connectivity/reachability is central.
Topological Sort	There are dependencies and an ordering must respect them.
DSU	You need dynamic connectivity/component merging.
Trie	Prefix or word lookup is central.
Bit Manipulation	Binary representation/XOR/masks create a simpler state.
DP	The problem has overlapping subproblems and an optimal substructure/state transition.
Advanced Graph	Weighted paths, MST, SCC, bridges or specialized graph structure matters.
Advanced Data Structures	Fast range queries/updates or strict operation guarantees are required.

7. Stage 4 — The 100 Core Product-Based Problems

These are the first serious problem set after the beginner stage. Solve them in order of the patterns below. Difficulty is intentionally mixed so that you build recognition rather than only Easy-problem confidence.

Arrays / Hashing

- 1. Two Sum
- 2. Contains Duplicate
- 3. Valid Anagram
- 4. Group Anagrams
- 5. Majority Element
- 6. Best Time to Buy and Sell Stock
- 7. Maximum Subarray
- 8. Product of Array Except Self
- 9. Longest Consecutive Sequence
- 10. Subarray Sum Equals K

Two Pointers / Sliding Window

- 11. Valid Palindrome
- 12. Two Sum II - Input Array Is Sorted
- 13. 3Sum
- 14. Container With Most Water
- 15. Trapping Rain Water
- 16. Longest Substring Without Repeating Characters
- 17. Longest Repeating Character Replacement
- 18. Permutation in String
- 19. Minimum Size Subarray Sum
- 20. Minimum Window Substring

Prefix Sum / Binary Search

- 21. Running Sum of 1d Array
- 22. Range Sum Query - Immutable
- 23. Continuous Subarray Sum
- 24. Binary Search
- 25. Search Insert Position
- 26. Find First and Last Position of Element in Sorted Array
- 27. Search in Rotated Sorted Array
- 28. Find Minimum in Rotated Sorted Array
- 29. Koko Eating Bananas
- 30. Capacity To Ship Packages Within D Days

Linked List

- 31. Reverse Linked List
- 32. Merge Two Sorted Lists
- 33. Linked List Cycle
- 34. Middle of the Linked List
- 35. Remove Nth Node From End of List

- 36. Add Two Numbers
- 37. Reorder List
- 38. Copy List with Random Pointer
- 39. Sort List
- 40. Merge k Sorted Lists

Stack / Queue

- 41. Valid Parentheses
- 42. Min Stack
- 43. Evaluate Reverse Polish Notation
- 44. Daily Temperatures
- 45. Next Greater Element I
- 46. Car Fleet
- 47. Largest Rectangle in Histogram
- 48. Implement Queue using Stacks
- 49. Rotting Oranges
- 50. Sliding Window Maximum

Trees / BST

- 51. Maximum Depth of Binary Tree
- 52. Same Tree
- 53. Invert Binary Tree
- 54. Diameter of Binary Tree
- 55. Binary Tree Level Order Traversal
- 56. Validate Binary Search Tree
- 57. Kth Smallest Element in a BST
- 58. Lowest Common Ancestor of a Binary Tree
- 59. Binary Tree Right Side View
- 60. Binary Tree Maximum Path Sum

Heap / Intervals / Greedy

- 61. Kth Largest Element in an Array
- 62. K Closest Points to Origin
- 63. Top K Frequent Elements
- 64. Find Median from Data Stream
- 65. Merge Intervals
- 66. Insert Interval
- 67. Non-overlapping Intervals
- 68. Meeting Rooms II
- 69. Jump Game
- 70. Gas Station

Graphs

- 71. Number of Islands
- 72. Clone Graph
- 73. Course Schedule

- 74. Course Schedule II
- 75. Pacific Atlantic Water Flow
- 76. Surrounded Regions
- 77. Graph Valid Tree
- 78. Number of Connected Components in an Undirected Graph
- 79. Word Ladder
- 80. Network Delay Time

Backtracking / Trie / Bits

- 81. Subsets
- 82. Permutations
- 83. Combination Sum
- 84. Word Search
- 85. N-Queens
- 86. Implement Trie (Prefix Tree)
- 87. Design Add and Search Words Data Structure
- 88. Single Number
- 89. Number of 1 Bits
- 90. Missing Number

Dynamic Programming / Design

- 91. Climbing Stairs
- 92. House Robber
- 93. House Robber II
- 94. Coin Change
- 95. Word Break
- 96. Longest Increasing Subsequence
- 97. Longest Common Subsequence
- 98. Edit Distance
- 99. Unique Paths
- 100. LRU Cache

100-problem gate: Re-solve every failed problem. Then take a random subset without seeing the pattern name. If you cannot start independently, the set is not finished.

8. Stage 5 — Full Pattern Mastery Problem Banks

Arrays

Two Sum • Best Time to Buy and Sell Stock • Maximum Subarray • Product of Array Except Self • Maximum Product Subarray • 3Sum • 4Sum • Longest Consecutive Sequence • Set Matrix Zeroes • Spiral Matrix • Rotate Image • First Missing Positive • Trapping Rain Water

Hashing

Valid Anagram • Group Anagrams • Top K Frequent Elements • Longest Consecutive Sequence • Subarray Sum Equals K • 4Sum II • Find All Anagrams in a String • Minimum Window Substring • LRU Cache

Two Pointers

Valid Palindrome • Two Sum II • 3Sum • Container With Most Water • Trapping Rain Water • Sort Colors • Boats to Save People • Valid Palindrome II

Sliding Window

Longest Substring Without Repeating Characters • Longest Repeating Character Replacement • Permutation in String • Find All Anagrams in a String • Minimum Size Subarray Sum • Fruit Into Baskets • Minimum Window Substring • Sliding Window Maximum • Subarrays with K Different Integers

Prefix Sum

Range Sum Query • Subarray Sum Equals K • Continuous Subarray Sum • Subarray Sums Divisible by K • Maximum Size Subarray Sum Equals k • Contiguous Array • Count of Range Sum

Binary Search

Binary Search • Search Insert Position • First/Last Position • Search Rotated Sorted Array • Find Minimum Rotated Array • Find Peak Element • Koko Eating Bananas • Capacity to Ship Packages • Split Array Largest Sum • Median of Two Sorted Arrays

Linked List

Reverse Linked List • Merge Two Sorted Lists • Linked List Cycle • Middle of Linked List • Remove Nth Node • Add Two Numbers • Reorder List • Copy Random List • Sort List • Merge k Sorted Lists • Reverse Nodes in k-Group

Stack

Valid Parentheses • Min Stack • RPN • Daily Temperatures • Next Greater Element • Car Fleet • Decode String • Largest Rectangle in Histogram • Maximal Rectangle • Basic Calculator

Queue/Deque

Queue using Stacks • Circular Queue • Recent Calls • Rotting Oranges • Sliding Window Maximum • Shortest Subarray with Sum at Least K • Jump Game VI

Recursion

Fibonacci • Power • Recursive Binary Search • Tree Traversals • Generate Parentheses • Different Ways to Add Parentheses

Backtracking

Subsets • Subsets II • Permutations • Permutations II • Combinations • Combination Sum • Letter Combinations • Generate Parentheses • Palindrome Partitioning • Word Search • N-Queens • Sudoku Solver • Word Search II

Trees

Maximum Depth • Same Tree • Invert Tree • Symmetric Tree • Diameter • Balanced Tree • Level Order • Zigzag Level Order • Right Side View • LCA • Construct Tree • Serialize/Deserialize • Maximum Path Sum • House Robber III

BST

Validate BST • Search BST • Insert BST • Delete BST • LCA BST • Kth Smallest • Recover BST • Trim BST • Range Sum BST • Two Sum IV

Heap

Kth Largest • Last Stone Weight • K Closest Points • Top K Frequent • Task Scheduler • Median from Data Stream • Merge k Sorted Lists • IPO • Furthest Building

Intervals

Merge Intervals • Insert Interval • Non-overlapping Intervals • Meeting Rooms II • Minimum Arrows • Interval List Intersections • Employee Free Time • My Calendar I/II/III • Minimum Interval to Include Each Query

Greedy

Assign Cookies • Jump Game • Jump Game II • Gas Station • Partition Labels • Task Scheduler • Candy • Hand of Straights • Boats to Save People • Minimum Number of Taps

Graphs

Path Exists • Number of Islands • Flood Fill • Max Area of Island • Clone Graph • Rotting Oranges • Pacific Atlantic • Surrounded Regions • Connected Components • Graph Valid Tree • Course Schedule • Word Ladder • Reconstruct Itinerary • Is Graph Bipartite

Topological Sort

Course Schedule • Course Schedule II • Alien Dictionary • Parallel Courses • Recipes from Supplies

DSU

Graph Valid Tree • Redundant Connection • Accounts Merge • Connected Components • Number of Islands II • Most Stones Removed

Trie

Implement Trie • Add and Search Words • Replace Words • Longest Word • Word Search II • Maximum XOR

Bits

Single Number • Number of 1 Bits • Counting Bits • Power of Two • Missing Number • Sum of Two Integers • Bitwise AND • Single Number II/III • Maximum XOR

DP 1D

Climbing Stairs • Min Cost Climbing Stairs • House Robber • House Robber II • Decode Ways • Coin Change • Coin Change II • Word Break • Perfect Squares • Partition Equal Subset Sum • Target Sum

DP Grid

Unique Paths • Unique Paths II • Minimum Path Sum • Triangle • Maximal Square • Dungeon Game • Minimum Falling Path Sum • Cherry Pickup • Longest Increasing Path in a Matrix

DP Subsequences/Strings

LIS • Number of LIS • LCS • Edit Distance • Distinct Subsequences • Shortest Common Supersequence • Longest Palindromic Subsequence • Interleaving String • Wildcard Matching • Regular Expression Matching

Advanced DP

Burst Balloons • Palindrome Partitioning II • Stone Game • Partition Array for Maximum Sum • Minimum Cost to Cut a Stick • Stock IV • Super Egg Drop • Bitmask DP • Tree DP • Digit DP

Advanced Graphs

Dijkstra • Bellman-Ford • Floyd-Warshall • 0-1 BFS • MST/Kruskal • MST/Prim • Critical Connections • SCC/Tarjan • Eulerian Path

Advanced Data Structures

Segment Tree • Fenwick Tree • Sparse Table • Range Minimum Query • Range Maximum Query • LRU Cache • LFU Cache • All O(1) Data Structure

9. Company-Style Final Preparation

- Random Easy: solve quickly and explain complexity.
- Random Medium: 30–40 minute timed attempt.
- Random Hard: focus on decomposition and communication; do not panic if you need a hint.
- Blind pattern test: no tags, no topic name, no editorial.
- Mixed 2-problem session: one familiar pattern + one unseen pattern.
- Weekly contest: Codeforces or another timed platform.
- Company-specific practice: use current company-tagged problems close to the interview period.
- Mock interview: explain before coding, think aloud, test edge cases, and discuss trade-offs.
- Re-solve old failures until the same mistake stops recurring.

10. Interview Communication Checklist

- Clarify constraints and input/output.
- Give a small example.
- State the brute-force idea.
- Explain why brute force may be too slow.
- Propose the optimized data structure/pattern.
- State expected time and space complexity before coding.
- Code in small, testable steps.
- Run through normal and edge cases.
- Explain trade-offs and alternatives.
- Do not silently code for the entire interview.

11. Your Anti-AI Rule

Level 0: Solve yourself → **Level 1:** debugger/console → **Level 2:** documentation → **Level 3:** search for conceptual clarification → **Level 4:** one hint → **Level 5:** approach review → **Level 6:** solution only when necessary. If you see the solution, close it, re-code from memory, and schedule a re-solve.

12. Definition of “Ready”

You are ready for serious product-based DSA interviews when you can repeatedly solve representative Medium problems, identify patterns without labels, explain complexity, code in JavaScript under time pressure, recover from bugs, and handle unfamiliar variants. The 100 problems are your first major milestone—not the finish line.

Recommended long-term target: 300–400 high-quality interview problems + repeated re-solves + company-specific practice + contests + mocks.

13. Final Rule

Do not chase problem count.

Finish JavaScript → finish beginner problems → finish complexity → finish each pattern → finish the 100 → expand to 300–400 → go blind/random → company-specific → mock interviews.

One thing. Master it. Then move. ■